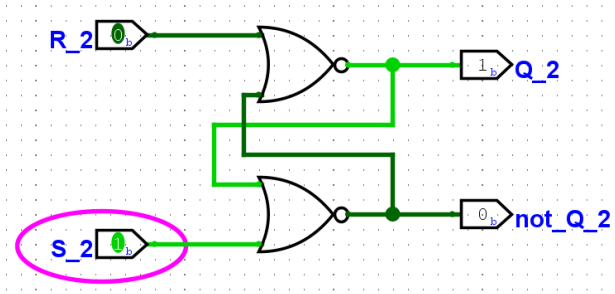
1. RS Latch

The outputs of two NOR gates are cross-coupled, each feeding the input of the other.

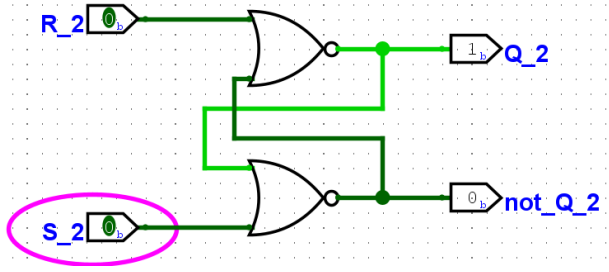
When S=R=1, both Q and Q^- become 0 and their complementary relationship is broken, so this input is forbidden.

S	R	Q	Operation
1	0	1	Set
0	0	Previous	Hold
0	1	0	Reset
1	1	0 (Q^-=0)	Forbidden

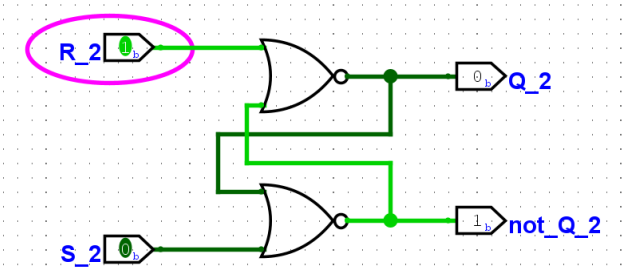
1-1 Logisim (NOR gates)



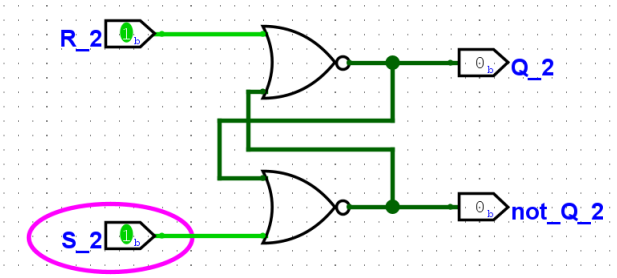
[1-1] Set: S=1, R=0 → Q=1



[1-2] Hold: S=R=0 → Q=1 held



[1-3] Reset: S=0, R=1 → Q=0



[1-4] Forbidden: S=R=1 → Q=Q^-=0

1-2 VHDL

The two NOR equations are used to build the same structure as the gate circuit.

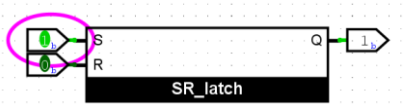
```vhdl
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY SR_latch IS
PORT (
     S,R : IN    STD_LOGIC;
     Q   : OUT   STD_LOGIC
);
END SR_latch;

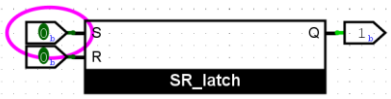
ARCHITECTURE bhv OF SR_latch IS
SIGNAL notQ : STD_LOGIC;
SIGNAL Q_temp : STD_LOGIC;
BEGIN

Q    <= Q_temp;
Q_temp  <= R nor notQ;
notQ <= S nor Q_temp;
end bhv;
```

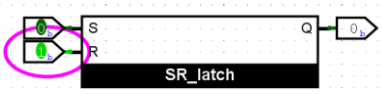
[2-0] SR_latch VHDL code



[2-1] Set → Q=1



[2-2] Hold → Q=1 held



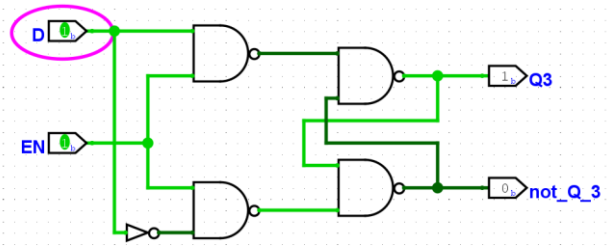
[2-3] Reset → Q=0

2. D Latch

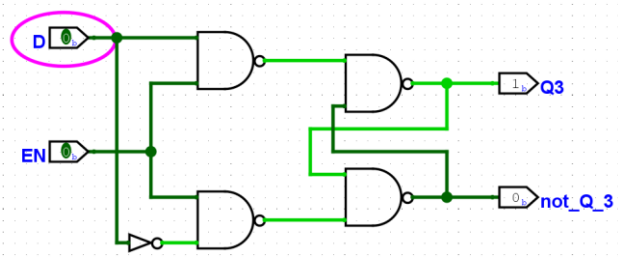
Built from four NAND gates and one inverter. Because D and its complement are used as the two inputs, the forbidden S=R=1 state cannot occur.

EN	D	Q
1	0	0
1	1	1
0	x	Previous value held

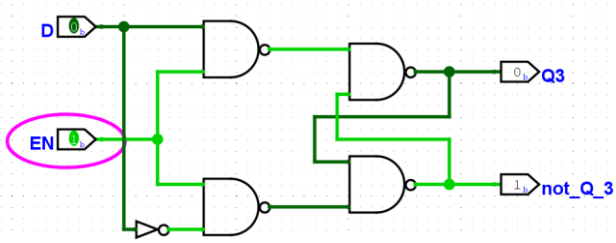
2-1 Logisim (NAND gates)



[3-1] EN=1, D=1 → Q=1



[3-2] EN=0 → Q=1 held even though D=0



[3-3] EN=1, D=0 → Q=0

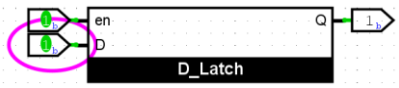
2-2 VHDL

`en='1'`: only then is `Q <= D` executed.

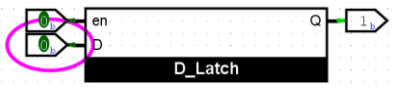
Since there is no else branch, the previous value is held when en=0.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY D_Latch IS
5  PORT (
6       en : IN std_logic;
7       D  : IN std_logic;
8       Q  : OUT std_logic
9  );
10 END D_Latch;
11
12 ARCHITECTURE bhv OF D_Latch IS
13 BEGIN
14
15      PROCESS(en,D)
16      BEGIN
17           IF (en = '1') THEN
18                Q <= D;
19
20           END IF;
21
22      END PROCESS;
23 END bhv;
```

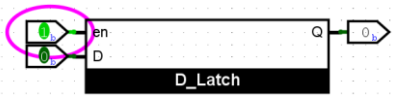
[4-0] D_Latch VHDL code



[4-1] EN=1, D=1 → Q=1



[4-2] EN=0 → Q=1 held



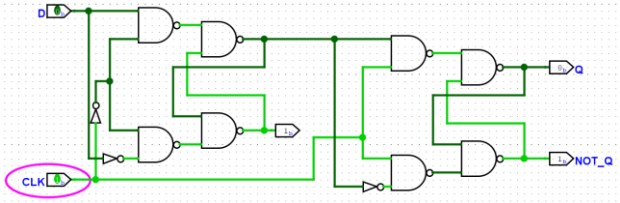
[4-3] EN=1, D=0 → Q=0

3. D Flip-Flop

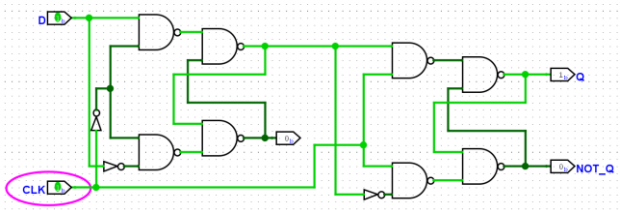
A master-slave structure made of two D latches (master and slave) and an inverter. The master latch operates on the complement of CLK (through the inverter) and the slave latch on CLK. When CLK=0, the master is open and captures D, while the slave is closed and holds Q. At the moment CLK changes from 0 to 1, the master closes and the slave opens, passing the master value to Q, so Q is updated only on the rising edge.

CLK	D	Q
↑ (rising edge)	0	0
↑ (rising edge)	1	1
Held at 0 or 1	x	Previous value held

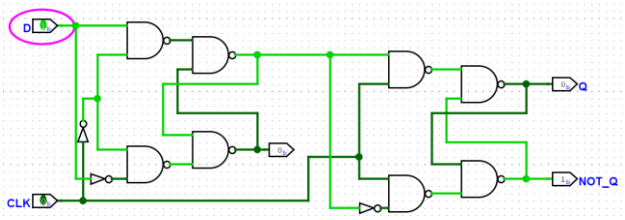
3-1 Logisim (master-slave, NAND)



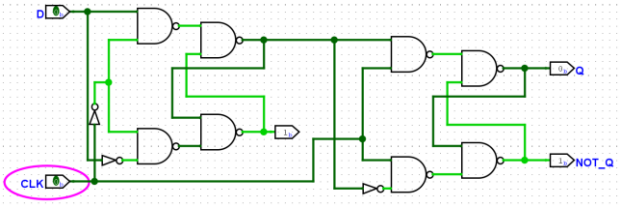
[5-1] CLK=1, D=0 → Q=0



[5-2] CLK=1, D=1 → Q=1 (rising edge)



[5-3] CLK=0, D=1 → only master changes, Q=0 held



[5-4] CLK=0, D=0 → Q=0 held

The middle output pin is the inverted output of the master latch. In [5-3], when D=1 arrives while CLK=0, the master value changes but the slave is closed, so Q stays 0. Only after the clock rises does Q become 1, as in [5-2].

3-2 VHDL

`clk'event AND clk='1'` (rising edge): only then is `Q <= D` executed, so Q changes only on the rising edge.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY D_FlipFlop IS
PORT (
     clk : IN std_logic;
     D   : IN std_logic;
     Q   : OUT std_logic
);
END D_FlipFlop;


ARCHITECTURE bhv OF D_FlipFlop IS

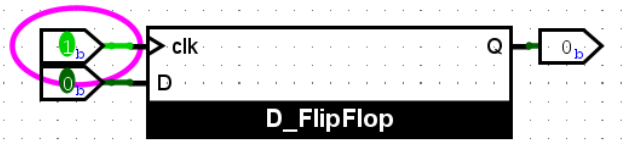
BEGIN

     PROCESS(clk)
     BEGIN
          IF (clk'event AND clk = '1') THEN
               Q <= D;

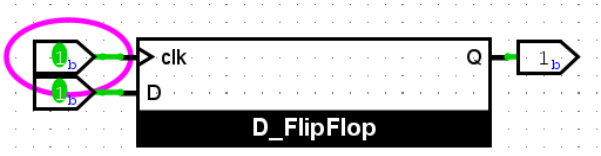
          END IF;

     END PROCESS;
END bhv;
```

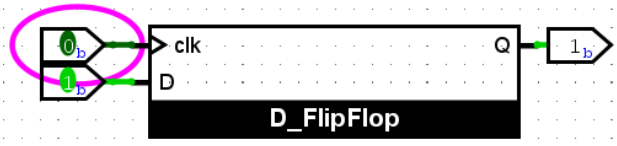
[6-0] D_FlipFlop VHDL code



[6-1] clk↑, D=0 → Q=0



[6-2] clk↑, D=1 → Q=1



[6-3] clk=0, D=1 → Q=1



[6-4] clk=0, D=0 → Q=1 held

4. JK Flip-Flop

A master-slave JK flip-flop built from eight NAND gates.

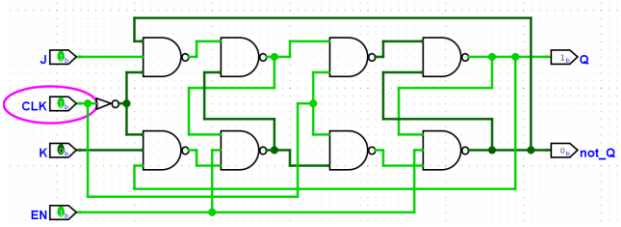
The input NAND gates receive the outputs Q^- and Q as feedback together with J and K, which implements Toggle.

The complement of CLK is fed to the master and CLK to the slave, so Q changes on the rising edge.

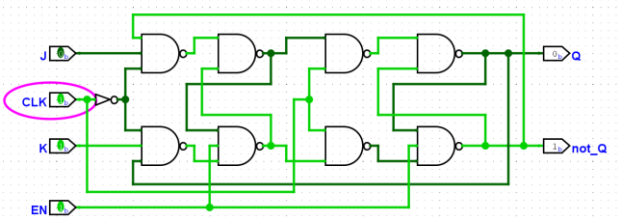
When EN=0, the inputs are blocked and the value is held.

J	K	Q(next)	Operation
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q^-	Toggle

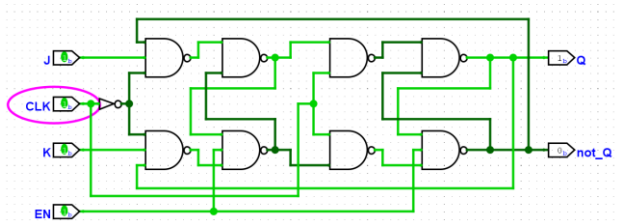
4-1 Logisim (NAND + feedback)



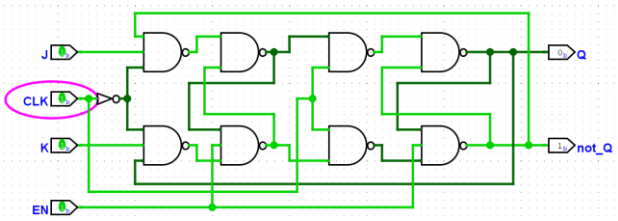
[7-1] J=1, K=0 → Q=1 (Set)



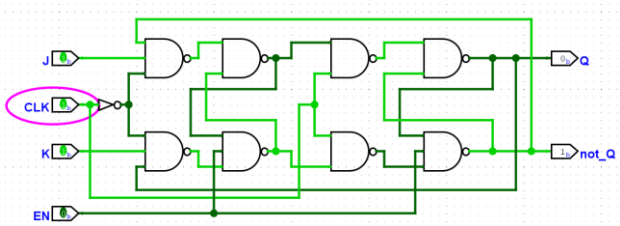
[7-2] J=0, K=1 → Q=0 (Reset)



[7-3] J=K=1, 1st clock → Q=1



[7-4] J=K=1, 2nd clock → Q=0 (Toggle)



[7-5] EN=0 → Q held even , J=K=1

4-2 VHDL

On the rising edge, Set/Reset/Toggle is performed according to the J, K combination; otherwise (J=K=0) the value is held. Asynchronous `clr`, `set` inputs were also added; clr is checked first, so it has higher priority.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY JK_FlipFlop IS
PORT (
     clr : IN std_logic;
     set : IN std_logic;
     clk : IN std_logic;
     J   : IN std_logic;
     K   : IN std_logic;
     Q   : OUT std_logic
);
END JK_FlipFlop;


ARCHITECTURE bhv OF JK_FlipFlop IS
SIGNAL Q_temp : std_logic := '0';

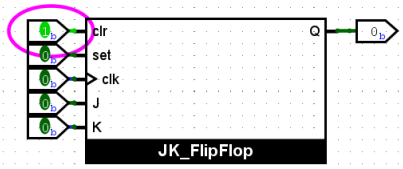
BEGIN
     Q <= Q_temp;
     PROCESS(clk,clr,set)
     BEGIN
          IF clr = '1' THEN    -- CLEAR (RESET)
               Q_temp <= '0';
          ELSIF set = '1' THEN    -- SET
               Q_temp <= '1';

          ELSIF (clk'event AND clk = '1') THEN
               IF (J = '1' AND K = '0') THEN    -- SET
                    Q_temp <= '1';
               ELSIF (J = '0' AND K = '1') THEN -- RESET
                    Q_temp <= '0';
               ELSIF (J = '1' AND K = '1') THEN -- TOGLE
                    Q_temp <= NOT Q_temp;

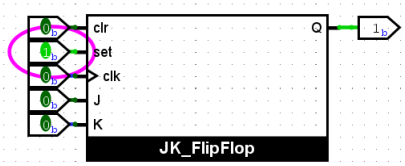
               END IF;

     END PROCESS;
END ARCHITECTURE bhv;
```

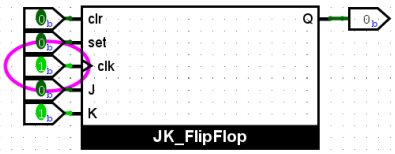
[8-0] JK_FlipFlop VHDL code



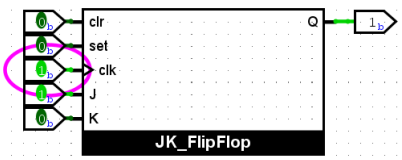
[8-1] clr=1 → Q=0



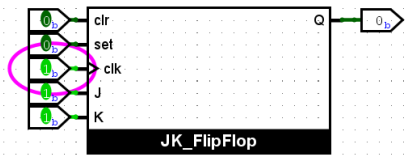
[8-2] set=1 → Q=1



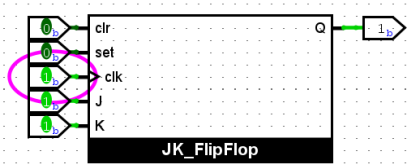
[8-3] J=0, K=1, clk↑ → Q=0



[8-4] J=1, K=0, clk↑ → Q=1



[8-5] J=K=1, clk↑ → Q=0



[8-6] J=K=1, next clk↑ → Q=1 (Toggle)

5. D Flip-Flop with Set / Clear

The code checks clr and set before checking the clock edge. Therefore clr and set take effect immediately regardless of the clock (asynchronous), and because clr is checked first in the if statement, Clear has priority over Set.

When both signals are 0, the circuit operates as a normal rising-edge D flip-flop.

clr	set	clk	Q
1	x	x	0 (immediate)

0	1	x	1 (immediate)
0	0	↑	D
0	0	Held	Previous value held

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY D_FlipFlop_clr_set IS
PORT (
     clk : IN std_logic;
     clr : IN std_logic;
     set : IN std_logic;
     D   : IN std_logic;
     Q   : OUT std_logic
);
END D_FlipFlop_clr_set;


ARCHITECTURE bhv OF D_FlipFlop_clr_set IS

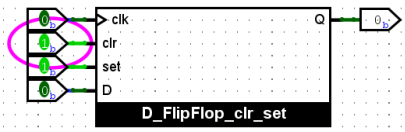
BEGIN

     PROCESS(clk,clr,set)
     BEGIN
          IF clr = '1' THEN
               Q <= '0';
          ELSIF set = '1' THEN
               Q <= '1';
          ELSIF (clk'event AND clk = '1') THEN
               Q <= D;

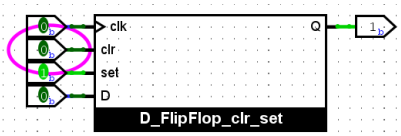
          END IF;

     END PROCESS;
END bhv;
```

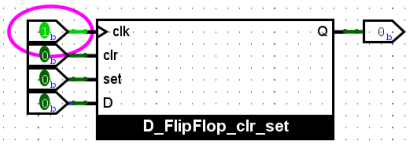
[9-0] D_FlipFlop_clr_set VHDL code



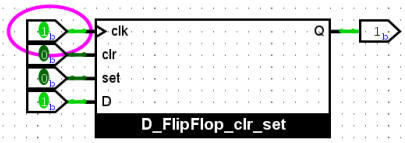
[9-1] clr=1, set=1 → Q=0



[9-2] set=1 → Q=1



[9-3] clk↑, D=0 → Q=0



[9-4] clk↑, D=1 → Q=1



[9-5] clk=0, D=0 → Q=1 held